Enterprise Web Application Assignment 5

Generics & Utility Classes: UCS553

Q1. Generic Method for Calculating Array Sum

Write a Java program to create a generic method that calculates the sum of an array of numeric values. Use a bounded type parameter such as T extends Number and the doubleValue() method to calculate the sum. Test the method using arrays of integers, floats, and doubles.

1. Question1.java

```
package q1;

public class Question1 {

    public static <T extends Number> double calculateSum(T[] numbers) {
        double sum = 0.0;
        for (T num : numbers) {
            sum += num.doubleValue();
        }
        return sum;
    }

    public static void main(String[] args) {
        System.out.println("=== Generic Sum Calculation ===");

        Integer[] intArr = { 10, 20, 30, 40 };
        Float[] floatArr = { 1.5f, 2.5f, 3.5f };
        Double[] doubleArr = { 10.25, 20.75, 30.5 };

        System.out.println("Integer array sum: " + calculateSum(intArr));
        System.out.println("Float array sum: " + calculateSum(floatArr));
        System.out.println("Double array sum: " + calculateSum(doubleArr));
    }
}
```

Q1 Output

Terminal: Q1 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp bin
q1.Question1
=== Generic Sum Calculation ===
Integer array sum: 100.0
Float array sum: 7.5
Double array sum: 61.5
```

Q2. Student Email Storage and Handling using Optional

Write a Java program that stores a student's email address in an Optional<String> object. Perform the following operations:
1. Create an Optional containing an email address.
2. Create an empty Optional.
3. Display the email using ifPresent().
4. Use orElse() to provide a default email address.
5. Use orElseThrow() to handle a missing email value.

1. Question2.java

```
package q2;

import java.util.Optional;

public class Question2 {

    public static void main(String[] args) {
        System.out.println("=== Student Email with Optional ===");

        Optional<String> email = Optional.of("shreyaas@thapar.edu");
        Optional<String> emptyEmail = Optional.empty();

        System.out.print("1. Using ifPresent(): ");
        email.ifPresent(e -> System.out.println("Student Email: " + e));

        System.out.print("2. Using ifPresent() on empty: ");
        emptyEmail.ifPresent(e -> System.out.println("Student Email: " + e));
        System.out.println("(No output because email is empty)");

        System.out.println("3. Using orElse() on present email: " + email.orElse("notfound@thapar.edu"));
        System.out.println("4. Using orElse() on empty email: " + emptyEmail.orElse("default@thapar.edu"));

        System.out.println("5. Using orElseThrow():");
        try {
            String val = email.orElseThrow(() -> new IllegalArgumentException("Email is not available"));
            System.out.println("Present value: " + val);
        } catch (Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }

        try {
            String val = emptyEmail.orElseThrow(() -> new IllegalArgumentException("Email address is missing"));
            System.out.println("Empty value: " + val);
        } catch (Exception e) {
            System.out.println("Caught expected exception: " + e.getMessage());
        }
    }
}
```

Q2 Output

Terminal: Q2 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp bin
q2.Question2
=== Student Email with Optional ===
1. Using ifPresent(): Student Email: shreyaas@thapar.edu
2. Using ifPresent() on empty: (No output because email is empty)
3. Using orElse() on present email: shreyaas@thapar.edu
4. Using orElse() on empty email: default@thapar.edu
5. Using orElseThrow():
Present value: shreyaas@thapar.edu
Caught expected exception: Email address is missing
```

Q3. Generic Student Result with Input Validation

Create a generic class Result<T extends Number> to store marks of a student. The program should:
1. Store marks using a generic type.
2. Calculate the percentage or average using numeric conversion methods.
3. Determine whether the student has passed based on a specified threshold.
4. Display the result using Optional if the marks are available.
5. Use Objects.requireNonNull() to validate required inputs.

1. Question3.java

```
package q3;

import java.util.Objects;
import java.util.Optional;

class Result<T extends Number> {
    private String studentName;
    private T[] marks;

    public Result(String studentName, T[] marks) {
        this.studentName = Objects.requireNonNull(studentName, "Student name cannot be null");
        this.marks = Objects.requireNonNull(marks, "Marks array cannot be null");
    }

    public double calculateAverage() {
        if (marks.length == 0) return 0.0;
        double total = 0.0;
        for (T mark : marks) {
            total += mark.doubleValue();
        }
        return total / marks.length;
    }

    public boolean isPassed(double threshold) {
        return calculateAverage() >= threshold;
    }

    public void displayResult(double threshold) {
        Optional<T[]> optionalMarks = Optional.ofNullable(marks);
        if (optionalMarks.isPresent()) {
            double avg = calculateAverage();
            boolean passed = isPassed(threshold);
            System.out.println("Student: " + studentName);
            System.out.print("Marks: ");
            for (T m : marks) {
                System.out.print(m + " ");
            }
            System.out.println();
            System.out.println("Average: " + avg);
            System.out.println("Status: " + (passed ? "Passed" : "Failed"));
        } else {
            System.out.println("Marks not available for student: " + studentName);
        }
    }
}

public class Question3 {
    public static void main(String[] args) {
        System.out.println("=== Student Result with Generics ===");

        Integer[] intMarks = { 85, 90, 78, 88 };
        Result<Integer> r1 = new Result<>("Shreyaas", intMarks);
        r1.displayResult(50.0);

        System.out.println();
        Double[] doubleMarks = { 45.5, 48.0, 42.5 };
        Result<Double> r2 = new Result<>("Aman", doubleMarks);
        r2.displayResult(50.0);

        System.out.println("\nTesting Objects.requireNonNull():");
        try {
            new Result<Integer>(null, intMarks);
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: " + e.getMessage());
        }

        try {
            new Result<Integer>("Rahul", null);
        } catch (NullPointerException e) {
            System.out.println("Caught NullPointerException: " + e.getMessage());
        }
    }
}
```

Q3 Output

Terminal: Q3 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp bin
q3.Question3
=== Student Result with Generics ===
Student: Shreyaas
Marks: 85 90 78 88
Average: 85.25
Status: Passed

Student: Aman
Marks: 45.5 48.0 42.5
Average: 45.333333333333336
Status: Failed

Testing Objects.requireNonNull():
Caught NullPointerException: Student name cannot be null
Caught NullPointerException: Marks array cannot be null
```

Q4. Event Management using LocalDate and LocalTime

Write a Java program to manage an event using LocalDate and LocalTime. The program should:
1. Accept an event name, event date, and start time.
2. Display the event details.
3. Check whether the event date is in the future, today, or in the past.
4. Calculate the number of days remaining until the event.
5. Calculate the event's end time if the event lasts for two hours.
6. Use Optional<String> to handle an optional venue name and display "Venue not specified" when it is absent.

1. Question4.java

```
package q4;

import java.time.LocalDate;
import java.time.LocalTime;
import java.time.temporal.ChronoUnit;
import java.util.Optional;

class Event {
    private String name;
    private LocalDate date;
    private LocalTime startTime;
    private Optional<String> venue;

    public Event(String name, LocalDate date, LocalTime startTime, String venueName) {
        this.name = name;
        this.date = date;
        this.startTime = startTime;
        this.venue = Optional.ofNullable(venueName);
    }

    public void displayDetails() {
        System.out.println("Event Name: " + name);
        System.out.println("Date: " + date);
        System.out.println("Start Time: " + startTime);
        System.out.println("Venue: " + venue.orElse("Venue not specified"));

        LocalTime endTime = startTime.plusHours(2);
        System.out.println("End Time (2 hours duration): " + endTime);

        LocalDate today = LocalDate.now();
        if (date.isAfter(today)) {
            long daysRemaining = ChronoUnit.DAYS.between(today, date);
            System.out.println("Status: Upcoming event (" + daysRemaining + " days remaining)");
        } else if (date.isBefore(today)) {
            System.out.println("Status: Past event");
        } else {
            System.out.println("Status: Happening today!");
        }
    }
}

public class Question4 {
    public static void main(String[] args) {
        System.out.println("=== Event Management System ===");

        LocalDate futureDate = LocalDate.now().plusDays(15);
        LocalTime startTime = LocalTime.of(10, 0);

        Event e1 = new Event("Tech Symposium", futureDate, startTime, "Auditorium A");
        System.out.println("\nEvent 1:");
        e1.displayDetails();

        LocalDate pastDate = LocalDate.now().minusDays(5);
        LocalTime startPast = LocalTime.of(14, 30);
        Event e2 = new Event("Coding Contest", pastDate, startPast, null);
        System.out.println("\nEvent 2:");
        e2.displayDetails();

        Event e3 = new Event("Web Workshop", LocalDate.now(), LocalTime.of(16, 0), "Lab 3");
        System.out.println("\nEvent 3:");
        e3.displayDetails();
    }
}
```

Q4 Output

Terminal: Q4 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp bin
q4.Question4
=== Event Management System ===

Event 1:
Event Name: Tech Symposium
Date: 2026-11-04
Start Time: 10:00
Venue: Auditorium A
End Time (2 hours duration): 12:00
Status: Upcoming event (15 days remaining)

Event 2:
Event Name: Coding Contest
Date: 2026-10-15
Start Time: 14:30
Venue: Venue not specified
End Time (2 hours duration): 16:30
Status: Past event

Event 3:
Event Name: Web Workshop
Date: 2026-10-20
Start Time: 16:00
Venue: Lab 3
End Time (2 hours duration): 18:00
Status: Happening today!
```

Q5. Generic Utility Class

Develop a generic utility class containing the following methods:
1. isEqual(T a, T b) - checks whether two objects are equal.
2. getDefault(Optional<T> value, T defaultValue) - returns the contained value or a default.
3. max(T a, T b) - returns the larger of two comparable objects.
4. isNull(Object obj) - checks whether an object is null using the Objects class.
Demonstrate the methods using strings, integers, and other suitable objects.

1. Question5.java

```
package q5;

import java.util.Objects;
import java.util.Optional;

class Utility {

    public static <T> boolean isEqual(T a, T b) {
        return Objects.equals(a, b);
    }

    public static <T> T getDefault(Optional<T> value, T defaultValue) {
        return value.orElse(defaultValue);
    }

    public static <T extends Comparable<T>> T max(T a, T b) {
        if (a.compareTo(b) >= 0) {
            return a;
        } else {
            return b;
        }
    }

    public static boolean isNull(Object obj) {
        return Objects.isNull(obj);
    }
}

public class Question5 {
    public static void main(String[] args) {
        System.out.println("=== Generic Utility Class Demo ===");

        System.out.println("\n1. Testing isEqual():");
        System.out.println("isEqual(10, 10): " + Utility.isEqual(10, 10));
        System.out.println("isEqual(\"Java\", \"Python\"): " + Utility.isEqual("Java", "Python"));
        System.out.println("isEqual(null, null): " + Utility.isEqual(null, null));

        System.out.println("\n2. Testing getDefault():");
        Optional<String> name = Optional.of("Shreyaas");
        Optional<String> noName = Optional.empty();
        System.out.println("getDefault(name, \"Guest\"): " + Utility.getDefault(name, "Guest"));
        System.out.println("getDefault(noName, \"Guest\"): " + Utility.getDefault(noName, "Guest"));

        Optional<Integer> score = Optional.empty();
        System.out.println("getDefault(score, 0): " + Utility.getDefault(score, 0));

        System.out.println("\n3. Testing max():");
        System.out.println("max(25, 42): " + Utility.max(25, 42));
        System.out.println("max(\"Apple\", \"Banana\"): " + Utility.max("Apple", "Banana"));
        System.out.println("max(3.14, 2.71): " + Utility.max(3.14, 2.71));

        System.out.println("\n4. Testing isNull():");
        String s1 = null;
        String s2 = "Hello";
        System.out.println("isNull(null): " + Utility.isNull(s1));
        System.out.println("isNull(\"Hello\"): " + Utility.isNull(s2));
    }
}
```

Q5 Output

Terminal: Q5 Output

```
shreyaas@shreyaass-MacBook-Air Enterprise Web Applications % /opt/homebrew/opt/openjdk/bin/java -cp bin
q5.Question5
=== Generic Utility Class Demo ===

1. Testing isEqual():
isEqual(10, 10): true
isEqual("Java", "Python"): false
isEqual(null, null): true

2. Testing getDefault():
getDefault(name, "Guest"): Shreyaas
getDefault(noName, "Guest"): Guest
getDefault(score, 0): 0

3. Testing max():
max(25, 42): 42
max("Apple", "Banana"): Banana
max(3.14, 2.71): 3.14

4. Testing isNull():
isNull(null): true
isNull("Hello"): false
```